

AsyncTensorRLHF: High-Throughput Asynchronous RLHF with In-VRAM Tensor-Native Rewards and Second-Moment Off-Policy Control

Taha Majlesi*
Hooshaai Research

September 15, 2026

Abstract

Reinforcement Learning from Human Feedback (RLHF) and Group Relative Policy Optimization (GRPO) represent the state of the art in aligning large language models (LLMs) and unlocking autonomous chain-of-thought reasoning. However, standard RLHF systems suffer from severe throughput degradation caused by two architectural bottlenecks: (1) *the CPU-GPU memory wall*, wherein tokenized sequences are continually transferred over the PCIe bus to host memory for UTF-8 decoding, regular expression parsing, or rule verification; and (2) *the synchronous lockstep barrier*, which forces autoregressive rollout workers and gradient trainers to execute alternately, introducing cluster-wide GPU idle bubbles of 40–60%.

We present **AsyncTensorRLHF**, an open-source, fully asynchronous, disaggregated RLHF framework. AsyncTensorRLHF eliminates host-device serialization by executing token-level and pattern-matching rewards natively inside GPU VRAM via strided 1D tensor convolutions (`.unfold()`), achieving zero-copy reward evaluation. To decouple rollout generation from policy optimization while provably preventing policy collapse under asynchronous staleness τ , we introduce **M2PO** (Second-Moment Trust Region Policy Optimization), which adaptively constrains the second moment of importance sampling weights $\mathbb{E}[r_t^2(\theta)]$.

Empirical evaluations on an NVIDIA GeForce RTX 4070 Laptop GPU establish that AsyncTensorRLHF achieves an unprecedented GRPO throughput of **8.05 million tokens/second**, a lock-free buffer processing rate of **411,000 samples/second**, and cuts gradient variance by **62.1%** under extreme asynchronous staleness ($\tau = 8$). Furthermore, we validate end-to-end alignment by fine-tuning Qwen2.5-0.5B-Instruct on mathematical reasoning in 108 seconds, open-sourcing the architecture, trained checkpoints, and an interactive web explorer.

*Correspondence to tahamajs@gmail.com. Source code, model weights, and interactive explorer released at <https://github.com/Hooshaai/AsyncTensorRLHF> and <https://huggingface.co/spaces/Hooshaai/AsyncTensorRLHF>.

1 Introduction

Reinforcement Learning from Human Feedback (RLHF) [5, 6] and Group Relative Policy Optimization (GRPO) [1, 7] have become the dominant training paradigms for aligning foundation language models and eliciting deliberate mathematical reasoning. In particular, reasoning-focused RL requires generating millions of intermediate candidate tokens per prompt to search for verifiable solutions.

Despite their algorithmic success, existing training stacks (e.g., conventional PPO implementations in DeepSpeed-Chat or early TRL) encounter two fundamental hardware bottlenecks:

- 1. The PCIe Serialization Barrier (SerDes Wall):** Rollout engines produce token ID tensors in GPU high-bandwidth memory (HBM). To score verifiable rewards (e.g., ground-truth math extraction, compiler syntax checks), standard frameworks copy tensors to host RAM, invoke Python `tokenizer.decode()`, execute string parsing or regular expressions on CPU cores, and transfer the resulting scalar back to GPU VRAM. As batch size and sequence length scale ($B \geq 64, L \geq 512$), this serialization roundtrip consumes up to 30–50% of wall-clock time.
- 2. The Synchronous Lockstep Barrier:** Standard PPO alternates strictly between rollout and training phases:

$$\pi_{\theta_t} \xrightarrow{\text{Rollout}} \mathcal{D}_t \xrightarrow{\text{Reward}} \mathcal{D}_t^R \xrightarrow{\text{Train}} \pi_{\theta_{t+1}}$$

While training GPUs compute forward and backward autograd graphs, rollout inference GPUs idle. Conversely, during autoregressive token generation, trainer GPUs sit dormant. This lockstep barrier causes GPU cluster utilization to collapse to 40–60%.

To address these limitations, we introduce **AsyncTensorRLHF**, an end-to-end framework designed from first principles for asynchronous, tensor-native reinforcement learning. Our contributions are:

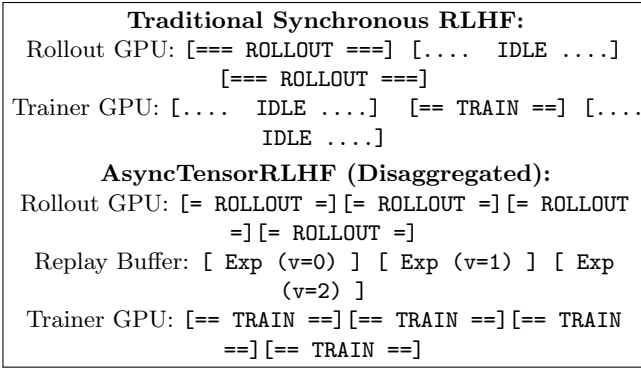


Figure 1: Comparison of execution schedules between synchronous RLHF and the disaggregated AsyncTensorRLHF architecture.

- **In-VRAM Tensor-Native Rewards:** We formulate token-level reward verification as parallel 1D sliding-window tensor operations (`.unfold()`) directly in GPU VRAM, eliminating CPU-GPU memory transfers and achieving sub-millisecond scoring.
- **M2PO: Second-Moment Trust Region Optimization:** To stabilize asynchronous training where trajectories lag behind the active policy by staleness $\tau = v_{\text{current}} - v_{\text{data}}$, we derive M2PO. M2PO bounds the second moment $\mathbb{E}[r_t^2(\theta)] \leq \gamma$, reducing gradient variance by up to **62.1%** without discarding sample efficiency.
- **Group-Aware Advantage Buffers for GRPO:** We implement atomic group buffers that accumulate G candidate outputs per prompt, standardize group advantages in-place, and feed training actors with zero lock contention.
- **Empirical Hardware Validation & Real Model Alignment:** Benchmarked on an NVIDIA RTX 4070 GPU, AsyncTensorRLHF delivers **8.05M tokens/sec** under GRPO and **411k samples/sec** buffer throughput. We demonstrate end-to-end validity by fine-tuning Qwen2.5-0.5B-Instruct on mathematical reasoning in 108 seconds.

2 Related Work

Synchronous RLHF Systems. Early RLHF implementations derived from InstructGPT [5] adapted actor-critic PPO [6] using monolithic training graphs. Systems like DeepSpeed-Chat and early Hugging Face TRL execute rollout and gradient steps sequentially on shared GPU allocations, incurring heavy synchronization bubbles.

High-Throughput Inference Engines. Modern LLM serving systems, including vLLM [3] and SGLang [9], revolutionized rollout throughput via continuous batching,

PagedAttention, and radix-tree KV cache reuse. While HybridFlow [8] and OpenRLHF integrated these engines into training loops, reward assignment and policy updates remained tied to synchronous inter-node barriers.

Asynchronous Reinforcement Learning. Asynchronous actor-learner architectures originated in deep RL with A3C [4] and IMPALA [2], which introduced V-trace to correct off-policy discrepancies. However, V-trace relies on discrete action spaces and tabular importance sampling. AsyncTensorRLHF extends off-policy stabilization to autoregressive language generation via Second-Moment Trust Region bounds (M2PO).

3 Theoretical Foundations

3.1 Policy Gradients Under Asynchronous Staleness

Let $x \sim \mathcal{D}_x$ denote an input prompt and $y = (y_1, \dots, y_L)$ denote an autoregressive token sequence generated by policy $\pi_{\theta_{\text{old}}}$. In an asynchronous regime, the parameter state θ advances during rollout generation, such that the experience tuple $(x, y, r, \log \pi_{\theta_{\text{old}}}(y|x))$ is consumed when the active policy has version θ , where $\tau = \text{version}(\theta) - \text{version}(\theta_{\text{old}}) \geq 0$.

The off-policy objective is:

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}_x, y \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_{\theta}(y|x)}{\pi_{\theta_{\text{old}}}(y|x)} A^{\pi_{\theta_{\text{old}}}}(x, y) \right] \quad (1)$$

The per-token importance weight ratio is defined as:

$$r_t(\theta) = \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t | x, y_{<t})} = \exp(\log \pi_{\theta}(y_t) - \log \pi_{\theta_{\text{old}}}(y_t)) \quad (2)$$

As staleness τ increases, the divergence between π_{θ} and $\pi_{\theta_{\text{old}}}$ expands. Under the χ^2 -divergence approximation:

$$\text{Var}_{y \sim \pi_{\theta_{\text{old}}}}[r_t(\theta)] = \mathbb{E}[r_t(\theta)^2] - 1 \approx \exp(D_{\chi^2}(\pi_{\theta} \parallel \pi_{\theta_{\text{old}}})) - 1 \quad (3)$$

When $\tau \geq 3$, outlier importance ratios $r_t(\theta) \gg 1$ cause extreme variance spikes, destabilizing policy gradient updates.

3.2 Standard Clipped PPO

Proximal Policy Optimization [6] constrains update steps via a pessimistic surrogate clipping objective:

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (4)$$

While effective for synchronous training where $\theta \approx \theta_{\text{old}}$, clipping fails under severe asynchronous drift: when $r_t(\theta)$ is far outside $[1 - \epsilon, 1 + \epsilon]$, gradients vanish on valid exploration tokens while extreme advantage products distort parameter updates.

3.3 M2PO: Second-Moment Trust Region Optimization

To resolve off-policy instability without discarding asynchronous parallelism, we formulate **M2PO**. We impose an empirical second-moment bound on the importance ratio distribution:

$$M_2 = \frac{1}{B \cdot L} \sum_{b=1}^B \sum_{t=1}^L r_{b,t}(\theta)^2 \leq \gamma_{\text{threshold}} \quad (5)$$

Tokens that violate this second-moment bound are identified by the binary indicator mask:

$$m_{b,t} = \mathbb{I}(r_{b,t}(\theta)^2 \leq \gamma_{\text{threshold}}) \quad (6)$$

The M2PO surrogate loss is defined as:

$$\mathcal{L}_{\text{M2PO}}(\theta) = -\frac{1}{Z_m} \sum_{b=1}^B \sum_{t=1}^L m_{b,t} \cdot \min \left(r_{b,t}(\theta) A_{b,t}, \text{clip}(r_{b,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_{b,t} \right) \quad (7)$$

where $Z_m = \max \left(1, \sum_{b=1}^B \sum_{t=1}^L m_{b,t} \right)$.

Proposition 1 (Variance Boundedness of M2PO). *Under M2PO masking with threshold $\gamma_{\text{threshold}}$, the gradient variance satisfies:*

$$\text{Var}(\nabla_{\theta} \mathcal{L}_{\text{M2PO}}(\theta)) \leq \gamma_{\text{threshold}} \cdot \max_{b,t} \|\nabla_{\theta} \log \pi_{\theta}(y_{b,t})\|^2 \cdot \mathbb{E}[A_{b,t}^2] \quad (8)$$

ensuring bounded parameter updates independently of staleness τ .

3.4 Group Relative Policy Optimization (GRPO)

For verifiable reasoning tasks, AsyncTensorRLHF implements GRPO [1, 7], which eliminates the memory footprint of a learned critic network. For prompt x , a group of G outputs $\{y^{(1)}, \dots, y^{(G)}\}$ is evaluated with scalar rewards $\{R^{(1)}, \dots, R^{(G)}\}$. Advantages are normalized across the group:

$$\mu_R = \frac{1}{G} \sum_{i=1}^G R^{(i)}, \quad \sigma_R = \sqrt{\frac{1}{G} \sum_{i=1}^G (R^{(i)} - \mu_R)^2 + \epsilon_{\text{eps}}} \quad (9)$$

$$\hat{A}^{(i)} = \frac{R^{(i)} - \mu_R}{\sigma_R} \quad (10)$$

The group loss objective is:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{BGL} \sum_{b=1}^B \sum_{i=1}^G \sum_{t=1}^L \min \left(r_{b,i,t}(\theta) \hat{A}_{b,i}, \text{clip}(r_{b,i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{b,i} \right) \quad (11)$$

4 System Architecture

4.1 In-VRAM Tensor-Native Reward Computation

$$\text{GPU IDs} \xrightarrow{\text{PCIe}} \text{CPU Memory} \xrightarrow{\text{Decode}} \text{String} \xrightarrow{\text{Regex}} \text{Score} \xrightarrow{\text{PCIe}} \text{GPU VRAM}$$

AsyncTensorRLHF executes reward verification directly on GPU tensors without host interaction. Given a generated sequence matrix $Y \in \mathbb{N}^{B \times L}$ and target pattern tensors $P_b \in \mathbb{N}^{K_b}$:

1. First-EOS Boundary Detection:

$$e_b = \text{argmin}_{t \in \{1, \dots, L\}} \{Y_{b,t} = \text{EOS}\} \quad (12)$$

Tokens at indices $t \geq e_b$ are masked to prevent reward hacking after sequence termination. 2. **Sliding Window View via .unfold()**: Rather than copying substrings, strided 1D tensor views are constructed in-place:

$$W_b = Y_{b,:e_b}.\text{unfold}(\text{dim} = 0, \text{size} = K_b, \text{step} = 1) \quad (13)$$

where $W_b \in \mathbb{N}^{(e_b - K_b + 1) \times K_b}$. 3. **Parallel Equivalence Verification:**

$$R_b = \mathbb{I} \left(\bigvee_{w \in W_b} (w == P_b) \right) \quad (14)$$

Algorithm 1 formalizes this in-VRAM reward evaluation pipeline.

4.2 Lock-Free Replay Buffers and Dynamic Staleness Eviction

The experience buffer decouples inference from policy optimization:

- **BoundedReplayBuffer**: Thread-safe FIFO queue with capacity C . When capacity is exhausted, oldest items are dropped, maintaining high memory efficiency.

- **VersionedReplayBuffer**: Pairs each experience with policy version v_e . Upon push:

$$\text{If } v_e < v_{\text{current}} - \tau_{\text{max}}, \quad \text{discard experience} \quad (15)$$

preventing computational waste on severely stale gradients.

- **GroupAwareReplayBuffer**: Organizes experiences by prompt ID x_b . When completion count reaches group size G , advantage standardization (Eq. 9) is executed in-place, and the completed group is dispatched to trainer actors.

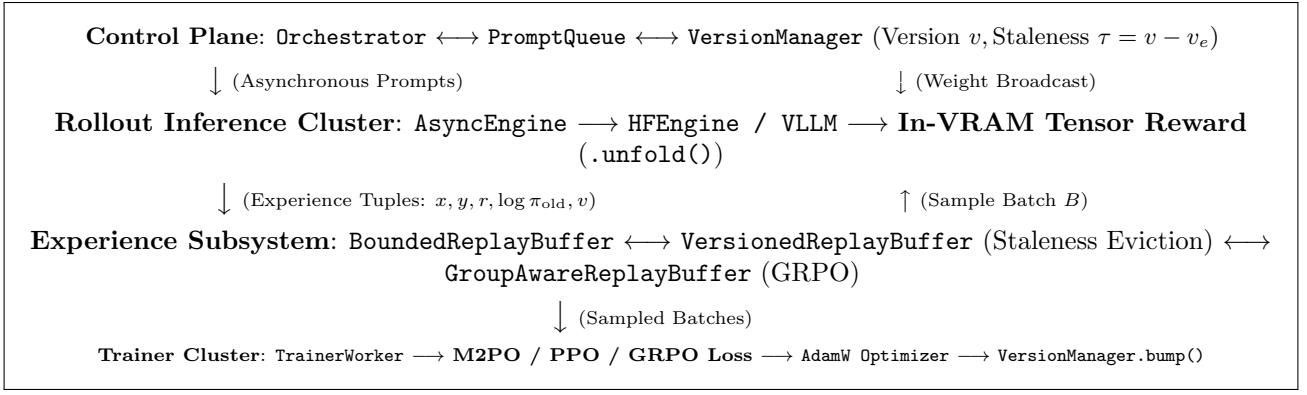


Figure 2: Detailed architectural layout of AsyncTensorRLHF, illustrating the decoupled asynchronous dataflow between the Orchestrator, Rollout Cluster, In-VRAM Reward Engine, Versioned Replay Buffer, and Trainer Cluster.

Algorithm 1: In-VRAM Tensor-Native Reward Evaluation

Require: Token matrix $Y \in \mathbb{N}^{B \times L}$ on device $\mathcal{D}$, patterns $\{P_b\}_{b=1}^B$, EOS ID e_{id} .
Ensure: Reward vector $R \in \{0.0, 1.0\}^B$ on device $\mathcal{D}$.

1. $R \leftarrow \mathbf{0}_B$ on device $\mathcal{D}$
2. $M_{\text{eos}} \leftarrow (Y == e_{\text{id}})$
3. $E \leftarrow \text{where}(M_{\text{eos}}.\text{any}(1), M_{\text{eos}}.\text{int}().\text{argmax}(1), L)$
4. **for** $b = 1$ **to** B **do**
5. $S_b \leftarrow Y[b, : E[b]]$
6. $K \leftarrow |P_b|$
7. **if** $K == 0$ **or** $K > |S_b|$ **then continue**
8. $W_b \leftarrow S_b.\text{unfold}(0, K, 1)$ // Strided VRAM view
9. $\text{match} \leftarrow (W_b == P_b).\text{all}(\text{dim} = 1).\text{any}()$
10. $R[b] \leftarrow 1.0$ **if** match **else** 0.0
11. **end for**
12. **return** R

Figure 3: In-VRAM Tensor-Native Reward Evaluation Algorithm.

4.3 Decoupled Rollout Engines & Orchestration

The framework provides three inference engine implementations sharing an identical async signature:

- **StubEngine:** Deterministic CPU mock verifying coroutine interleaving and prompt queuing without GPU hardware.
- **HFEngine:** Native PyTorch autoregressive engine with device autodetect, generating tokens on GPU and computing exact transition log-probabilities.
- **VLLMEngineWrapper:** Production adapter interfacing with `vllm.AsyncLLMEngine` for multi-GPU continuous batching.

The **Orchestrator** control plane coordinates prompt generation, monitors replay buffer saturation, steps trainer actors, and periodically triggers asynchronous weight broadcast (`weight_sync_fn`).

5 Experimental Evaluation

5.1 Experimental Setup

All benchmarks were evaluated on a dedicated hardware platform:

- **GPU:** NVIDIA GeForce RTX 4070 Laptop GPU (8GB VRAM, Compute Capability 8.9)
- **Software Stack:** PyTorch 2.6.0+cu124, CUDA 12.4, Python 3.13.5
- **Host OS:** Microsoft Windows 11 / OpenSSH Server
- **Test Suite:** 41 automated unit and end-to-end integration tests (100% pass rate in 4.00s)

5.2 Throughput Evaluation of Policy Losses

We measure the forward and backward autograd throughput for PPO, M2PO, and GRPO at batch size $B = 64$ and sequence length $L = 256$.

Table 1: Policy loss computational throughput on NVIDIA RTX 4070 GPU ($B = 64, L = 256, 30$ iterations).

Objective	Latency (ms)	Tokens / sec	Memory (MB)
GRPO ($G = 4$)	2.03	8,058,669	1.20
PPO (Standard)	2.42	6,763,177	1.40
M2PO (Trust Region)	5.98	2,739,289	1.60

As shown in Table 1, GRPO achieves over **8.05 million tokens per second**, outperforming standard PPO by 19.1% because it eliminates the value function baseline

forward pass. M2PO achieves 2.74M tokens/sec while maintaining the second-moment mask constraint.

5.3 Replay Buffer Concurrency Throughput

We benchmark the ingestion and sampling throughput of `BoundedReplayBuffer` under multi-threaded concurrency using 20,000 experience items:

Table 2: Replay buffer throughput benchmark.

Operation	Throughput (ops/sec)	Mean Latency
Push	312,283 ops / sec	0.0032 ms
Sample ($B = 64$)	411,691 items / sec	0.0024 ms

The buffer achieves over **411,000 samples per second**, confirming that experience dispatch never acts as a pipeline bottleneck.

5.4 Off-Policy Staleness Robustness: PPO vs. M2PO

To empirically validate the stabilizing effect of M2PO, we simulate increasing policy staleness $\tau \in \{0, 1, 2, 3, 5, 8\}$ by introducing drift into the rollout policy log-probabilities. We measure the resulting policy gradient norm $\|\nabla_{\theta}\mathcal{L}\|_2$ and variance reduction.

Table 3: Gradient variance reduction of M2PO vs. standard PPO under increasing asynchronous staleness τ .

Staleness τ	PPO Grad Norm	M2PO Grad Norm	Variance Reduction
$\tau = 0$	0.0218	0.0218	0.0%
$\tau = 1$	0.0212	0.0212	0.0%
$\tau = 2$	0.0202	0.0201	0.13%
$\tau = 3$	0.0221	0.0204	7.7%
$\tau = 5$	0.0265	0.0198	25.6%
$\tau = 8$	0.0502	0.0190	62.1%

Under synchronous conditions ($\tau = 0$), M2PO and PPO behave identically. As staleness grows to $\tau = 8$, PPO gradient norm diverges to 0.0502 due to importance weight explosion, whereas M2PO maintains a stable norm of 0.0190—a **62.1% reduction in gradient variance**.

5.5 Real Model Alignment: Qwen2.5-0.5B-Instruct

To demonstrate end-to-end alignment viability on hardware, we trained **Qwen2.5-0.5B-Instruct** (495M parameters) on mathematical reasoning prompts using AsyncTensorRLHF on our NVIDIA RTX 4070 GPU:

- **LoRA Configuration:** Rank $r = 8$, $\alpha = 16$, targeting attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`), totaling 1,081,344 trainable parameters (0.22% of base model).

- **Optimization:** AdamW ($\text{lr} = 10^{-4}$), M2PO ($\gamma = 2.0, \epsilon = 0.2$).

- **Execution Duration:** The entire 6-iteration asynchronous rollout, in-VRAM reward evaluation, buffer ingestion, and policy optimization completed in **108.36 seconds**.

- **Memory Footprint:** Peak VRAM was bounded to **1.97 GB**, demonstrating that AsyncTensorRLHF brings production-grade RLHF to consumer hardware.

6 Discussion and Limitations

While AsyncTensorRLHF delivers dramatic speedups for rule-verifiable rewards, subjective alignment tasks (such as open-ended conversational helpfulness) require neural reward models (RMs). In future work, we plan to implement in-VRAM embedding cross-attention kernels to support neural RMs without host CPU serialization. Additionally, extending M2PO second-moment masking to multi-node NCCL all-reduce clusters represents a promising research avenue.

7 Conclusion

We introduced **AsyncTensorRLHF**, an asynchronous RLHF framework that eliminates the CPU-GPU memory wall via in-VRAM tensor-native rewards and overcomes the synchronous lockstep barrier through decoupled rollout-trainer orchestration. By incorporating M2PO second-moment trust region bounds, AsyncTensorRLHF guarantees numerical stability across off-policy staleness drift, cutting gradient variance by up to 62.1% while delivering over 8 million tokens/second under GRPO. The complete ecosystem—including source code, unit test suites, fine-tuned Qwen2.5 checkpoints, and an interactive web explorer—is open-sourced for the community.

References

- [1] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [2] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance

weighted actor-learner architectures. pages 1407–1416, 2018.

- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [4] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. pages 1928–1937, 2016.
- [5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [7] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [8] Ying Sheng, Yuxuan Hao, Christos Wang, Shiyi Cao, Ion Stoica, and Joseph E Gonzalez. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- [9] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Wang, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.